

Level/Subject : **Mobile Programming**
Topic : Hooks
Week : 4
Activity : Building a Simple Attendance App using React Native
Allocated time : 120 mins labs
Due date : end of week

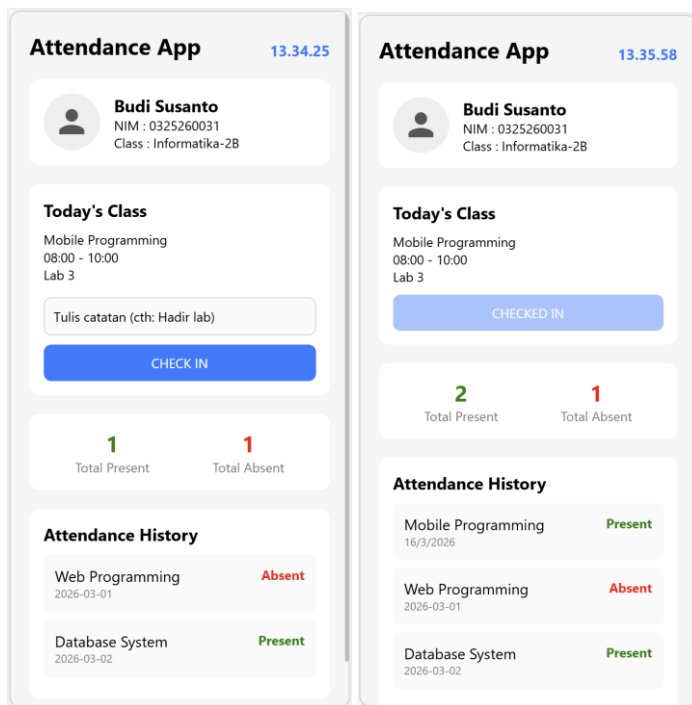
Introduction :

In this practical session, we will modify a simple Attendance Application using React Native.

During this practical session, we will focus on functional components & Hooks in React Native such as:

- `useMemo`
- `useRef`

The final application will look similar to the following interface.



Studi Kasus:

Mengubah Aplikasi Presensi dengan Optimasi Performa (`useMemo`) dan Referensi UI (`useRef`)

Tujuan Pembelajaran:

1. Memahami masalah *re-render* berlebih akibat *state* yang sering berubah (seperti jam digital).
2. Menggunakan `useMemo` untuk mencegah komputasi berat dijalankan berulang kali.
3. Menggunakan `useRef` untuk mengontrol elemen UI (seperti `<TextInput>`) secara langsung tanpa memicu *re-render*.

Langkah 1: Import Tambahan Hooks

Buka kembali file `Home.js`. Tambahkan `useMemo` dan `useRef` pada baris *import* paling atas, serta tambahkan komponen `TextInput` dari `react-native`.

Ubah baris *import* menjadi:

```
JS Home.js x
1  import React, {
2      useState,
3      useEffect,
4      useMemo,
5      useRef} from "react";
6  import { View, Text, SafeAreaView, StyleSheet,
7      TouchableOpacity,
8      ScrollView,
9      FlatList,
10     Alert,
11     TextInput} from 'react-native';
12 import { MaterialIcons } from '@expo/vector-icons';
```

Langkah 2: Menambahkan State Catatan & Referensi (useRef)

Di dalam komponen `Home`, tepat di bawah *state* jam digital, kita tambahkan *state* untuk teks catatan dan sebuah referensi kosong (`useRef`) yang nanti akan kita "tempelkan" ke elemen input.

```
28  // 3. STATE UNTUK JAM DIGITAL
29  const [currentTime : string , setCurrentTime] = useState( initialState: 'Memuat jam...');
30
31  // 4. STATE & REF UNTUK CATATAN (Baru)
32  const [note : string , setNote] = useState( initialState: '');
33  const noteInputRef : MutableRefObject<null> = useRef( initialValue: null); // Membuat "kait" kosong untuk UI
```

Langkah 3: Menghitung Statistik dengan useMemo

Kita perlu menghitung berapa kali mahasiswa hadir dan absen dari data historyData.

Penting: Karena layar kita me-re-render setiap detik (gara-gara jam digital), jika kita menggunakan fungsi hitung biasa, HP akan menghitung ulang ribuan data setiap detiknya! Di sinilah `useMemo` menyelamatkan performa aplikasi kita.

Tambahkan kode ini di bawah inisialisasi state/ref:

```
34
35 // 5. OPTIMASI KOMPUTASI DENGAN useMemo
36 const attendanceStats : {totalAbsent: number, totalPresent: number} = useMemo( factory: () : {totalAbsent: number, totalPresent: number} => {
37   // Teks ini hanya akan tercetak di terminal HANYA jika historyData bertambah,
38   // BUKAN setiap detik saat jam berubah!
39   console.log("Menghitung ulang statistik kehadiran...");
40
41   const presentCount : number = historyData.filter(item : {...} => item.status === 'Present').length;
42   const absentCount : number = historyData.filter(item : {...} => item.status === 'Absent').length;
43
44   return { totalPresent: presentCount, totalAbsent: absentCount };
45 }, deps: [historyData]); // <-- Dependencies: Hanya hitung ulang kalau 'historyData' berubah
46
```

Langkah 4: Memodifikasi Logika Check-In (Menggunakan useRef)

Sekarang kita ubah fungsi `handleCheckIn`. Kita wajibkan mahasiswa mengisi kolom catatan (misal: "Hadir tepat waktu"). Jika kosong, alih-alih hanya memberi `Alert`, kita gunakan `noteInputRef.current.focus()` untuk langsung menyorot kolom input tersebut dan memunculkan *keyboard* secara otomatis.

```
62 // FUNGSI LOGIKA ABSEN (Diperbarui)
63 const handleCheckIn = () : void => {
64   if (isCheckedIn) {
65     Alert.alert( title: "Perhatian", message: "Anda sudah melakukan Check In.");
66     return;
67   }
68
69   // Validasi Catatan menggunakan useRef
70   if (note.trim() === '') {
71     Alert.alert( title: "Peringatan", message: "Catatan kehadiran wajib diisi!");
72     noteInputRef.current.focus(); // <-- Sihir useRef: Memaksa kursor pindah ke input
73     return;
74   }
75
76   const newAttendance : {...} = {
77     id: Date.now().toString(),
78     course: "Mobile Programming",
79     date: new Date().toLocaleDateString( locales: 'id-ID'),
80     status: "Present",
81     // Anda juga bisa menambahkan properti catatan ke objek jika ingin ditampilkan
82   };
83
84   setHistoryData( value: [newAttendance, ...historyData]);
85   setIsCheckedIn( value: true);
86   Alert.alert( title: "Sukses", message: `Berhasil Check In pada pukul ${currentTime}`);
87 }
```

Langkah 5: Memperbarui Antarmuka (Menampilkan Input & Statistik)

Pada bagian return, kita akan menambahkan sebuah `<TextInput>` yang dikaitkan dengan `ref={noteInputRef}`, dan sebuah kotak kecil untuk menampilkan hasil hitungan `useMemo`.

Modifikasi bagian `classCard` milik `Today's Class` menjadi seperti ini:

```
125     { /* Today's Class */ }
126     <View style={styles.classCard}>
127         <Text style={styles.subtitle}>Today's Class</Text>
128         <Text>Mobile Programming</Text>
129         <Text>08:00 - 10:00</Text>
130         <Text>Lab 3</Text>
131
132         { /* Fitur Baru: Kolom Input Catatan dengan useRef */ }
133         { !isCheckedIn && (
134             <TextInput
135                 ref={noteInputRef} // <-- Menempatkan referensi ke elemen ini
136                 style={styles.inputCatatan}
137                 placeholder="Tulis catatan (cth: Hadir lab)"
138                 value={note}
139                 onChangeText={setNote}
140             />
141         ) }
142
143         <TouchableOpacity
144             style={[styles.button, isCheckedIn ? styles.buttonDisabled : styles.buttonActive]}
145             onPress={handleCheckIn}
146             disabled={isCheckedIn}
147         >
148             <Text style={styles.buttonText}>
149                 {isCheckedIn ? "CHECKED IN" : "CHECK IN"}
150             </Text>
151         </TouchableOpacity>
152     </View>
153
154     { /* Fitur Baru: Statistik Kehadiran (Hasil useMemo) */ }
155     <View style={styles.statsCard}>
156         <View style={styles.statBox}>
157             <Text style={styles.statNumber}>{attendanceStats.totalPresent}</Text>
158             <Text style={styles.statLabel}>Total Present</Text>
159         </View>
160         <View style={styles.statBox}>
161             <Text style={styles.statNumber, { color: 'red' }}>{attendanceStats.totalAbsent}</Text>
162             <Text style={styles.statLabel}>Total Absent</Text>
163         </View>
164     </View>
165
166     { /* Attendance History */ }
167     <View style={styles.classCard}>
168         <Text style={styles.subtitle}>Attendance History</Text>
169
170         <FlatList
171             data={historyData} // <-- Ubah 'history' menjadi 'historyData'
172             keyExtractor={(item) => item.id}
173             renderItem={renderItem}
174             scrollEnabled={false}
175         />
176     </View>
177 </ScrollView>
```

Langkah 6: Tambahan Styling Akhir

Tambahkan kode ini ke bagian bawah `StyleSheet` agar kolom input dan kotak statistiknya terlihat rapi dan selaras dengan desain W2.

```
285 // agar kolom input dan kotak statistiknya terlihat rapi dan selaras dengan desain
286 inputCatatan: {
287     borderWidth: 1,
288     borderColor: '#ccc',
289     borderRadius: 8,
290     padding: 10,
291     marginTop: 15,
292     backgroundColor: '#fafafa',
293 },
294 statsCard: {
295     flexDirection: 'row',
296     justifyContent: 'space-around',
297     backgroundColor: 'white',
298     padding: 15,
299     borderRadius: 10,
300     marginBottom: 20,
301 },
302 statBox: {
303     alignItems: 'center',
304 },
305 statNumber: {
306     fontSize: 24,
307     fontWeight: 'bold',
308     color: 'green',
309 },
310 statLabel: {
311     fontSize: 14,
312     color: 'gray',
313 },
```

Momen Pembuktian Saat Praktikum:

Jalankan aplikasi, buka Chrome/Expo Debugger dan perhatikan terminal (*Console*). Meskipun jam digital di layar HP terus berubah setiap detik (yang artinya layar sedang *re-render* ulang), tulisan Menghitung ulang statistik kehadiran... **TIDAK AKAN MUNCUL** setiap detik. Tulisan itu hanya akan muncul satu kali saat awal, dan satu kali lagi saat mereka sukses menekan tombol "Check In" (karena *array* `historyData` bertambah). Ini adalah pembuktian telak betapa pentingnya `useMemo` untuk menghemat napas *processor*!

Notes:

1. Create folder PRG6_M4_P3_XXXXXXXXXX
2. The folder must contain: contains
 - Project code without the `node_modules` folder
 - Screenshot of the application result
3. Zip the folder and submit it to the e-learning.